



金三银四面试突击ALL IN ONE 第一篇 —— 快人一步带你体验25K的一线高级前端开发的面试全流程

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 -- Typescript: JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue 应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂 React 最佳实践，手把手从 0 打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线 25K 前端 P6 岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节 T2-1 岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市 25K 前端 P6 岗的满分简历](#)

2. 今年的金三银四跟往年比有什么异同点？

面试

两极分化

1. (80%) 项目 当前经历、上份经历 角色定位 技术栈 业务场景 难点亮点 (1~5年)
 2. (20%) toB
 - a. 中小: (最佳实践 横向标准)
 - i. 最佳实践
 1. 前端全链路的方案实现 vite cli vue3 tailwind pinia router CI/CD 从每一行代码 -> 发布上线
 2. plugin analyze bundler
 3. components hooks utils ant design
 - ii. 横向标准
 4. scrm 代码质量 linter eslint commitlint prettier stylelint vite cra
 5. code review
 6. test jest
 7. components
 - b. 大: (专业度 垂类方向 技术方向)
 - i. 通用解决方案: toB CLI case + (定制化 vite)
 8. node: tool + BFF
 9. bundler: esbuild vite webpack 定制化 plugin loader
 10. 性能优化: 分析 -> 定位 -> 解决(打包产物优化 运行时优化)
 11. 稳定性: 监控 -> 治理
 - ii. 2D 3D formily low code monorepo
1. vue router 登录
 2. elemntUI antD 业务
 3. router json
 4. directive

1. 所谓的面试,是在一众简历中选择最合适的候选人。面试之所以卷,不是指简历的内容写的多或者项目多就好,而是在跟其他人竞争时,你有别人的简历中所没有的亮点,能够体现出你的技术水平,这才算好。所谓的行情不好,只是原先从简历中选择top 50%降低到了top 25%,但有亮点的简历,始终是在top 20%以内的,所以没有亮点和深度的简历之间越来越卷,越来越供大于求;但有亮点有技术深度的人始终是处于供不应求的情况;
2. 面试时,技术水平的高低始终是第一位的,如果水平足够,但学历低,优先级也会比学历高但水平低的人高;当然,如果水平一样,肯定有限学历高的候选人;
3. 现在的面试都是通过项目切入,根据项目中具体的实现的功能来引导使用的技术栈,考察的是针对对应的技术栈,具体掌握的深度和广度;
4. 面试的时间一般不会超过1小时,在1小时里不可能把所有的前端内容全部问完,所以面试官会挖掘候选人的项目亮点,通过候选人的面试,考察候选人对知识技能的掌握,再逐步提升难度,摸透候选人对知识掌握的边界;
5. 面试的过程是发掘出候选人优势的过程,不是打压候选人的过程,所以不用担心个别地方说的不好会不会导致面试不过,如果亮点项目很充分,能够体现出个人水平,面试官也不会在意一些边支末节的。

3. 如何做到当前面试的最优解?

女

28岁

汉

本科

上海

7年经验

13818416746

2429168074@qq.com

已婚已孕

技能特长

沟通能力90%

团队合作90%

责任感90%

求职意向

web前端上海

能力概述

HTML、CSS、JS、AJAX等基础知识；
性能优化、团队工程化搭建；
前端框架：React、Vue、微信小程序；
前端构建工具：webpack、Vite，基于业务定制脚手架；
版本管理：Git；
常见的设计模式、算法及网络协议；

工作经历

2020.9-2024.3

资深前端开发工程师

技术：React + React Router + Redux + git + 微信小程序

业务：负责sass产品微商城相关装修业务线

工作内容：基于微信公众号以及微信小程序生态圈进行定制化开发；熟练掌握团队内技术栈，能够快速完成具体需求的开发；负责相关性能优化、团队工程化搭建。

2016.6-2020.8

高级web开发工程师

技术：React + Vue + 微信小程序 + php

业务：负责核心云产品托管业务线、负责官网业务线

工作内容：从0~1搭建to B以及to C项目，负责团队技术选型、性能优化、代码质量管控。

项目经历

2020.9-2024.4

资深前端开发

项目介绍：
负责SASS产品的相关店铺装修、商品列表、商品分类等开发工作，用户访问量过亿。

简历.pdf

前端开发-3年开发经验

电话: 邮箱:
当前状态: 在职，正在找工作

能力概述

1. 良好前端开发基础，熟练使用h5，css3，js，ts进行前端开发；熟练使用axios，熟悉js运行机制，了解作用域链，熟悉this的机制；能够熟练的操作页面布局排版
2. 熟练使用Vue2和Vue3全家桶以及Element UI (plus) 等组件库，熟悉echarts等数据可视化组件库
3. 熟悉uni-app，并使用uni-app进行过跨平台项目开发
4. 熟练使用git，熟悉禅道，TAPD等项目管理工具，使用过Axure绘制原型图
5. 熟悉ES6语法，了解ES7-ES12相关特性
6. 熟悉tcp,udp,http,https,websocket等通信协议，了解防抖、节流，了解nginx
7. 熟悉web开发规范，代码管理相关规范，了解完整项目的开发流程，能与项目中各个角色顺畅沟通

工作经历

前端开发工程师

工作职责：
1. 负责项目功能的开发和优化，协调和管理项目开发流程，包括制定项目计划、跟踪项目进度、协调团队合作、解决问题等方面。
2. 负责编写和维护项目需求文档、开发文档和技术文档，确保团队成员能够理解需求和开发方案，并能够按照计划完成项目。

股份有限公司

前端开发工程师

工作职责：
1. 负责部门内项目的 Web 端开发及维护，构建用户界面，实现各种交互功能。
2. 对现有项目进行维护，保证项目的稳定运行。
3. 与项目相关方沟通，理解并梳理项目需求，编写详细的需求文档，以便于团队成员的理解和执行。
4. 根据需求文档，设计产品原型，使用原型设计工具如 Axure，制作出具有良好用户体验的产品原型。
5. 针对开发出的功能，编写测试用例，确保功能的正确性和稳定性。
6. 编写详细的使用手册，包括产品的功能介绍、操作步骤、注意事项等，方便用户使用和操作。
7. 在项目完成后，对项目进行验收，确保项目质量达到预期，按时交付给客户。

项目经历

监控告警推送系统

项目描述
该项目用于监控服务器的状态，根据告警的级别向相关工作人员进行告警推送，相关人员通过不同的告警内容解决现有问题。

个人职责
1. 对现有项目进行重构，包括优化代码结构、改进代码质量、提高代码可读性等方面。
2. 建设公共组件库，封装高级搜索、Alert、Confirm、Tabs等组件按照规范引入组件库，减少代码量。
3. 负责前端整体项目的功能开发，封装不同场景下需要使用的组件，统一组件风格。

项目实施
1. 优化复杂表单、大数据量查询界面，提高首屏加载速度从5s到0.5s，提升用户操作体验；
2. 对前端页面进行组件化开发，使用eslint + prettier + stylelint来统一项目代码风格。
3. 利用Echarts实现数据可视化，并针对大量数据渲染做了多层次优化
4. 对axios二次封装处理请求和响应，判断请求头是否携带token。对响应做过滤处理，处理不同状态码的返回
5. 负责对客户敏感信息脱敏显示、解密以及敏感操作二次验证等前端改造；
6. 使用 Vue Router 进行前端路由跳转利用了动态路由，导航守卫进行权限判断
7. 封装多语言插件、封装主题颜色切换插件

简历.pdf

LCP：largest content painting

分析了项目中不同页面的LCP，进行优化处理，提升了页面秒开率xxx 提升到了xxx

使用xxx完成xxx

定位：

- 问题什么 提升页面的秒开率，提升用户使用体感和页面性能
- 技术选型是什么
 - 定量指标分析：LCP
- 如何解决的 action
 - 分屏加载、分包、优化运行时
 - 加快接口 添加缓存
- 结果 xx -> xxx

通过上述的简历可以看到，只有在项目细节处通过具体的代码解释清楚，才能更好地表达出价值所在，诸如“优化用户体验交互，使用分布加载、分屏加载、异步加载的方式提升了页面首屏加载性能，从原本低端机2s+减少到1s内”这类的描述，只说了结果，要讲清楚过程实现，比如：

- 分布异步加载：使用lazy import 动态异步加载首页非核心模块；
- 优化用户体验交互：针对首页数据接口进行SWR缓存，加快首屏渲染时间；

3.1 探讨为什么面试只问八股文/ 我现在还没开始面试，八股文越看越没自信怎么办？

八股文准备不完怎么办

还是前面提到的，面试是面候选人的优点，而不是找缺点的过程。

所以，优先要做的是把简历项目中涉及到的知识点掌握完善，而不是一定要把所有的八股文全看完才敢放心面试，只要项目能够掌握熟练，出去面试就是水到渠成的过程。

面试过程中，在1小时以内的时间里，如果项目讲的好，足够吸引面试官的话，面试官是没有时间问八股文的内容的。所以要引导面试官问项目上的内容。

如果整个面试过程都是在询问项目，并且面试官问的问题都能很好的回答出来，这轮面试大概率过了。但如果面试官找不到项目亮点，问的八股文问题还回答不好，那这轮面试大概率是过不了的。

所以，在准备面试的过程中，建议先准备好项目，再在剩余的时间里抽空看看相关的八股文（正式课学员可以找班主任要完整的八股文资料）。

准备时都会，面试就卡壳

很多工作1~3年，甚至3~5年的同学都觉得自己对前端的内容都掌握了，但为什么一面试就挂呢？

绝大多数的同学是因为面试官在问项目中的具体实现时回答不出来导致的，只要面试的问题回答不上来，就说明水平够，看的懂跟讲的出来是两个不同的level的，看的懂只是你看了整体的代码，你的思路是跟着代码走的，但讲的出来意味着你对于代码的架构、需求分析是有自己的思考的，能够讲出来具体的实现过程。

所以，针对这类情况，建议把当前项目的代码完整阅读一遍。然后通过画图的方式把每个业务模块所包含的技术点绘制出来（如后台管理系统，登录模块、菜单管理模块、用户鉴权模块、页面布局模块等分别都涉及到哪些技术点，这些技术点是实现怎样的功能的），实现完整的架构图，在后续面试的时候，讲项目就是讲完整架构图的设计与实现原理。

在描述项目的过程中，可以遵循以下方式：

1. 项目背景及业务诉求：简要描述；
2. 技术选型：可以讲为什么要选择当前实现的这一套方案；
3. 技术方案：根据技术选型的方式，绘制架构图，详细讲架构图中不同模块的功能及最终实现的效果，要通过技术点描述出来，不要只说空话，实现了一个登录模块，重点是如何实现的登录模块；
4. 效果提升：通过这样一套架构，最终给业务带来了XXX的提升，最好有实际的量化数据体现出提升效果；

项目没有亮点

注意：一定不要强行给自己加戏，项目的亮点是从已有项目中挖掘出来的，而不是凭空捏造的。

很多同学觉得自己的项目没有亮点，就找网上的项目，结果网上找到的项目千篇一律，50%人的项目都是重复的，面试官自然连面试的机会都不会给你。

在课程中针对不同水平、不同技术栈的同学准备了对应的课程，可以通过VIP课程中对应的部分来完善自己的项目，最终一定要保证手写一遍代码，写完后，自然就能达到熟练地说出这些项目的具体实现过程和项目难点，因为难点就是在手写代码的时候不会点，掌握后就是你在这个项目中的亮点。

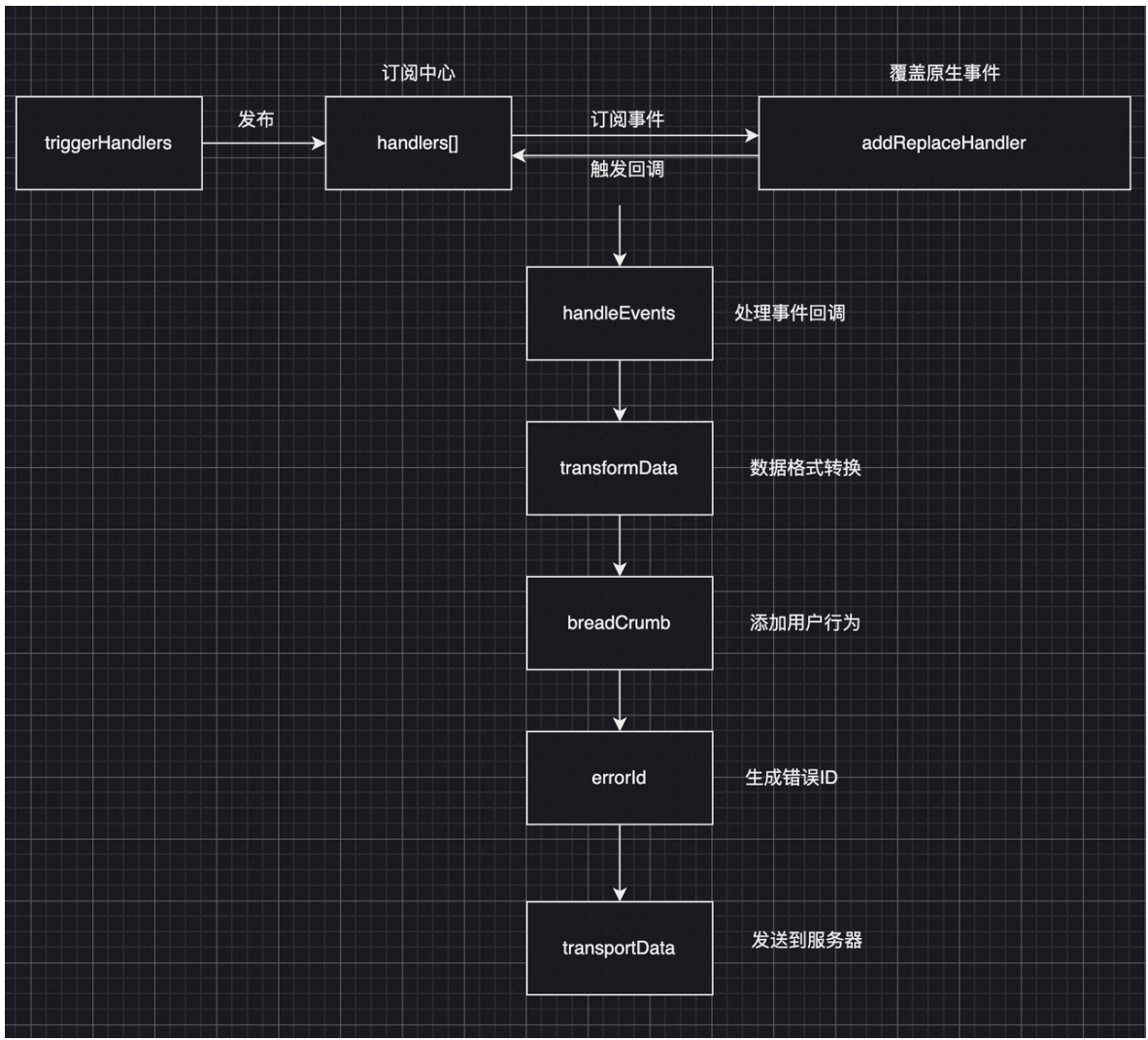
（正式课学员可以找班主任要完整的项目实战资料）

- Vue项目没有亮点：
 - Vue.js 源码与进阶
 - Vue2高级用法
 - Vue3新特性
 - Vue项目实战
 - 性能优化
 - Vue性能优化实战
- React项目没有亮点：
 - React.js 源码与进阶
 - React.js高级用法
 - 前端业务Hooks库
 - 性能优化
 - React性能优化实战
- 没有工程化实战经验：
 - JavaScript 高级内功修炼
 - JS 模块化详解
 - 工程化实战
 - 前端工程化详解&包管理工具
 - 模块化开发&webpack
- 没有算法经验：
 - 设计模式
 - 前端高频算法面试
 - 大厂算法真题讲解
- 没有前端统一的编码规范标准：
 - 实战项目1-前端编码规范工程化
- 没有稳定性实战经验
 - 实战项目2-前端稳定性监控

3.2 我的项目没什么可写的，我该怎么办？

3. 稳定性治理（类似参考实战课项目 [📖 8. 前端稳定性监控](#)）

- a. 描述清楚业务价值：通过监控收集前端各类错误信息，实现了支持多平台、可扩展、可插拔式的前端错误信息，通过日志上报，行为展示，从而快速定位问题，提升系统稳定性；
- b. 技术难点：定义好监控标准，支持插件化方式集成；抽离页面监控、Vue监控、React监控、小程序监控及性能监控；定义好插件统一收口，支持渐进式迭代升级；
- c. 体现亮点：
 - i. 定量收集不同错误、性能指标类型标准；
 - ii. 错误插件式集成流程；



- iii. 定义数据上报方式及时机：利用浏览器空余间隙上传 requestidlecallback，上传方式：XHR、Image及sendBeacon；
- iv. 定义统一报错日志；

3.1 不同年限准备面试的侧重点

针对目前的行情，市场对候选人水平的判定一般分为三个阶段：

3.1.1 1~2年内

这个阶段还属于成长期，更需要看重的是你的基础和热情。对于 JS 基础，计算机基础，网络通信，算法等部分的要求会相对高一些。毕竟这个阶段比较难考察你的业务项目中的沉淀，所以只能从基础部分入手考察。

- 在学校学习，或是利用网络上的各种资料巩固自己的基础，是这个阶段的关键；
- 在简历里用各种方式展示出你对前端的热情，让面试官看到你的潜力；
- 多去了解社区前沿技术，关注国内外的各种技术趋势；
- 尝试自己写一些小项目，或者是参与社区开源的项目；
- 开始记录自己的技术博客，用输出倒逼你的输入；

超出同水平的要求：

1. 能够回答出市场上绝大多数的八股文题目（正式课学员可以找班主任要完整的八股文资料）；
2. 能够讲清楚自己所做的项目使用的技术栈是什么，团队内前端技术的搭建落地包含哪些（类似：现在使用的技术栈是Vue + Pina + Vue Router + Vue-CLI 实现的，是如何具体使用这些框架的，使用这些框架的哪些工具实现了怎样的能力，注意：要具体到使用的API，可以忘记API的名称，但一定要支持使用这些API所达到的效果）；

3.1.2 2~5年

这个阶段一般来说是向着独当一面的工程师发展。也是非常关键的一个时期，避免一年的经验用三年。

- 社区里关于进阶的资料和路线有很多，平时多关注一下，补齐自己的基础知识；
- 平时常用的框架进阶一步去使用，比如它的一些高级用法是否有所掌握，有没有试着去了解它的原理实现；

- 日常的业务开发中不局限于完成功能，是否有去思考项目结构如何设计，如何封装基础工具，基础组件如何设计、开发、共享；
- 在日常的业务开发中有没有去思考团队提效的方式，比如：
 - 接入 eslint、prettier 等代码检验、风格统一的插件；
 - 工程化的角度思考本地开发的提效，如何去进行 webpack 构建的优化，最近社区 esbuild 很火，尝试去接入一下。vite 和 snowpack 的思路很赞，能不能在新项目中运用起来等等……
 - 平常如果经常有多项目开发的需求，整理出差异和统一的部分，建立团队内部的脚手架避免重复劳动。
 - 尝试搭建CI / CD 平台，尝试搭建npm 私服维护自己公司内部通用包；
 - 锻炼你的软技能，沟通协作也是很重要的一项能力。通过思考业务真实需求砍掉多余的需求，协调各个角色一起推进目标，也是高级工程师很重要的技能；

超出同水平的要求：

1. 能够回答出市场上绝大多数的八股文题目，并能够举一反三，讲出对应的原理及类似的框架；
2. 能够讲清楚现在所使用的框架的核心原理，项目中具体是如何落地的，为什么要这么配置，好处是什么（类似：现在实现了一个后台管理系统，使用的是Vue + Pina + Vue Router + Vue-CLI。其中，登录后用户的身份存储在前端哪里（如cookie），登录后不同的人展示菜单是如何实现的（通过Pina，针对用户身份进行全局状态的拦截处理），如何避免用户打开没有权限的页面（Vue Router 通过路由守卫实现针对不同类型的身份进行拦截）等，一定要讲出每个功能模块实现的细节和技术选型的思考在里面）；

3.1.3 5年以上

走到这个阶段，可能就往技术专家或者管理的方向前进了。我们希望你可以把握某（多）个具体产品或者技术方向的研发工作，独立负责一个复杂度高的项目，并突破其中的关键技术。

你需要具备相当的产品视野和技术深度，需要站在更宏观的角度来看问题，也需要具备一定的跨团队协作能力；能够制定所负责方向的产品和技术规划，并推动落地，同时在研发效率、质量、资源使用率、产品渗透率等方面有一定的提高：

- 如何负责技术调研，是否关注行业前沿趋势，根据不同场景选择最优的技术方案，能不能有拍板决定的能力和魄力；
- 技术经验是否丰富，有没有相当的技术储备，参与过的项目类型多吗，遇到的困难都是如何解决，是否有沉淀出一套自己的方法？拒绝一年的经验重复使用；
- 产品上是否能协助甚至主导业务目标的制定，并根据业务目标划分任务，制定排期，合理的推动项目达到预期效果；
- 是否带过团队，或者是协作过跨团队项目，带团队有什么心得，能协调处理团队成员情绪问题吗，成员技能分布不平衡等问题如何解决；
- 如何打造一个有技术氛围的团队，不局限于自己提升技术，而是帮助团队共同成长

超出同水平的要求：

1. 能够针对具体的业务场景，制定出完善的技术方案（比如在实际开发中，如何才能从0~1制定团队的前端编码规范的标准？如何制定优化前端的监控）（正式课学员可以找班主任要实战课录播的资料）；
2. 能够在自己的业务领域中，掌握相对前沿的技术实现，只要能够在以下类型中任意一个类型有深度的落地经验，能够完整讲出在当前团队内如何实现业界内相对领先的水平，这就是前端专家；

类型	体系职能
技术-前端-开发	负责人机交互层的界面开发，实现交互功能
技术-前端-架构	熟悉业务领域与前端技术发展，负责前端类库框架、研发流程等基础体系的设计并推动实现，负责业务领域的前端技术选型并推动落地
技术-前端-数据可视化	负责常规统计图表、业务洞察分析、地理空间数据分析等相关引擎、服务、产品及生态建设
技术-前端-Node	基于Node进行web服务开发或工具开发
技术-前端-图像互动	负责互动/游戏化业务，2D&3D图像渲染等研发工作
前端体验	1. 体验度量：通过设计体验模型来度量产品体验功能2. 体验优化：体验分析，如核心链路分析、体验验收&用户反馈等
前端工程化	1. 基础前端工程服务平台、前端上层链路研发平台、开发支撑平台2. 前端研发工具，如WebIDE etc
跨端技术	1. 跨软硬件设备，IoT、跨操作系统、跨App、跨渲染容器（最常见，如webView，weex，rn etc）2. 跨端生态
中后台技术	1. 基础UI组件，研发工具2. low Code、no Code逻辑编排 etc
前端智能化	1. D2C（Design to Code 设计稿转代码）2. 分析用户特征，推荐UI排版，千人千面
Serverless	1. 含义：Serverless = Server + less = 少/无服务器2. FaaS：Function as a service 代码+相关依赖+配置
数据可视化	1. 基础前端工程服务平台、前端上层链路研发平台、开发支撑平台2. 前端研发工具，如WebIDE etc
多媒体技术	1. 音视频基础、流媒体播放、视频剪辑等2. 多端（web、Hybrid、小程序）直播间

4. 具体技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



5. 2025年，前端 如何与 AI 结合

TLDR

- **对于单元测试：**AI 考虑了非常全面的分支条件，但是运行就报错，因为一些基础依赖、mock 方法、API 的使用不够准确；
- **对于代码生成：**第一次生成效果最好，在后续的讨论修改中逐渐变得上文不接下文，直到所有代码变得不可用；
- **对于代码补全：**Copilot 虽然很惊艳，但是在企业的业务开发过程中由于存在内部代码泄露风险，并不能随心所欲的使用；
- **对于代码助手：**向它寻求帮助时，它编造不存在工具和 API 推荐给我；

总的来说，AI 代码生成这件事经过了一年时间的考验，仿佛变成了食之无味弃之可惜的鸡肋，程序员战战兢兢使用 AI 生成着代码，却惊喜地发现这东西一时半会还替代不了自己。

在日常开发中，AI 已被业界许多工具证明可以提高开发效率。但如果代码量快速膨胀，人类还改的过来吗？**如果随着 AI 生产力进一步提升，当代码检查成为效率的瓶颈时，大量未经人类把关的 AI 代码直接用在生产环境，人类真的敢用吗？**

AI 的表现如何？

考虑到 AI 已经被鼓吹可以替代人类，那么人类程序员能做到的代码重构效果，AI 也一定能做到，所以一开始在提示词中希望它在重构的过程中优化文件结构，移除冗余代码，同时保持原始逻辑不变。这是一个非常“普通且自信”的想法：既要、又要、还要。

于是沟通遇到的第一个问题出现了，**即使将复杂的任务拆分，人类也很难用自然语言明确告知模型，怎么样算优化文件结构，什么样的逻辑是冗余逻辑**，毕竟人类程序员还要经过反复的需求讨论才能逐渐确认需求的全部细节。于是一步到位的提示词方案被否定。

接下来对 AI 的要求是：文件对文件，逻辑对逻辑进行代码转换，不必做过程中的优化和调整。这样任务就简单了许多，降低了沟通成本，指令的下达变得明确而高效。

随着项目的推进，又遇到了类似的问题：**对于混杂在多任务中的简单小任务，AI 无法做到，无论是明确定义、增加示例、反复强调、给予鼓励**。比如特定第三方组件库的使用参数，JSX 条件渲染的实现等。这时问题可能出现在这些小任务合集触碰到了 AI 能力的上限，**除非针对特定任务精调，去提升它在任务中的表现**。

去评判 AI 的表现，总的来说还是**沟通和能力边界**的问题。对于业务中使用的某个具体模型，它的能力边界不完全确定，由于模型内部逻辑原理的不可解释性，提示词工程师（Prompt Engineer，PE）们只能不断猜测、试验，而在这其中投入多少精力的度，是不确定的。另外考虑到 AI 的幻觉问题，如何评判“胜任”，如何确保 AI 每一次任务都“胜任”也是一个挑战。所以这充满不确定的沟通调试过程，影响了 AI 代码生成的效果，**较低的代码产物质量让它们暂时无法被信任地直接用在生产环境**。

总结

和人之间的信任一样，AI 的信任是逐渐建立的，从简单的任务到复杂的任务如果都能逐一胜任，那么人类对 AI 的信任就会增加。**对于 AI 出现的失误，并不是不被允许，只是这些失误需要是可解释的，并且有规律可规避的**。对 AI 更进一步的期待是，**其内部的运行逻辑是可解释的，其行为是可控的**。

我们期待这一天的到来，期待 AI 成为人类更加可信任的工作伙伴。

AI & 测试

你可能会觉得“啊，既然 AI 代码生成问题那么多，那么退一步让 AI 来测试吧”，但是，实际上 AI 测试当前也有着数不尽的问题。本小节将会详细拆解这些问题来逐一分析。

AI 与测试相关的问题

截止到目前，前端 + 大语言模型 + 测试的实践几乎为 0。如果你在 Github 搜索，仓库不超过 10 个，合计的 Star 数加起来 5 个都不到。

为什么？为什么感觉这么“前途无量”的场景连 Github 仓库都没几个？

这个问题很难用一句来解释清楚，但是我向你保证，看完本节，你一定会得到答案。

单元测试

先说实施起来最简单的单元测试。单元测试非常的直白，就是使用一段纯 JS 代码来测试另一段纯 JS 代码，比较结果，一样就成功不一样的就失败。这么简单的任务，强如 GPT-4，不会做不到吧？

能做到，但是不能完全做到。

其实在 AI + 单元测试，在 LLM 出现之前，业界就已经有很多很多的实践了，而在 LLM 后，又崛起了一批 LLM + 单元测试的试验品。但是我打赌读者一个也没听说过对不对？没听说过就对了，因为哪怕有了 LLM 的助力，效果依旧一言难尽，根本拿不出手。

接受率

在这里引入一个接受率的概念。简单来说就是 AI 生成了 100 个用例，哪些是可以直接使用的。假如有 80 个可以不经修改的直接使用，那么接受率就是 $80 / 100 = 80\%$ 。

但是这里有个非常尴尬事情——并不是说接受率 > 0% 那么就可以用，因为你依旧需要花精力在那些“不可用的用例”。

假如接受率是 40%，AI 生成了 100 个用例，你其实并不知道这 100 个用例中哪 40 个是正确的，而为了使用这 40 个正确的用例，你需要：

- 1. 想办法将可用的用例挑出来，并逐一验证
- 2. 将剩下不可用的用例一个一个改好
- 3. 重新整体检查，手动填补用例遗漏

假如接受率真的是 40%，这个过程将会是地狱，将远远超越你自己写 100 个用例的时间。

好了，那么不妨在此猜测一下，那么在 LLM 之前和之后，AI 单元测试的接受率是多少？

单元测试的进一步分析

通过接受率小节，我们了解的接受率必须高到一定阈值才有意义，否则就是单纯的把“写用例”变成了“调用例+改用例”。据我们的估计，接受率至少要达到 80% 以上，否则就完全是在帮倒忙。

- 而目前 LLM 单元测试 普遍的接受率仅为 50% 不到，而如果是用 NLP 来代替 LLM，接受率将低到 30% 以下：

Pass@1 [%]						Pass@10 [%]					
	c++	go	java	Python	Javascript		c++	go	java	Python	Javascript
code-gen-6b-multi	11%	13%	15%	18%	13%	code-gen-6b-multi	25%	22%	31%	30%	26%
codegeex-13b	16%	12%	20%	22%	15%	codegeex-13b	31%	25%	36%	39%	31%
code-genius-6b	18%	17%	19%	33%	17%	code-genius-6b	32%	31%	40%	50%	38%
code-genius-16b	TBD	24%	TBD	35%	TBD	code-genius-16b	TBD	34%	TBD	55%	TBD
GPT3-Davinci	48%	32%	40%	48%	49%	GPT3-Davinci	TBD	TBD	TBD	TBD	TBD

这就好比有个实习生，写了 10 个页面，其中：

- 1. 2 个页面看上去正常，实际上交互有各种问题和巧妙的 Bug
- 2. 4 个页面运行正常，但是和上面这种页面混在一起了，你需要测试后才能找到他们
- 3. 2 个页面打开就白屏，你简单修改几次它还是白屏，你也不知道具体为啥，需要慢慢调

4. 2 个页面忘做了

这种实习生你要吗？因此，AI & 单元测试依旧要求程序员拥有极强的单测完善能力，它不能代替你来直接完成单元测试。

其他测试

单元测试包含了太多的细节和边界情况，那么我们把视野拉远，比如 E2E 测试，是不是会好一点呢？正好前端单测用的也不多。别说，你还真别说，当我们聚焦于 E2E 而不是单测后，各种问题确实是迎刃.....增加了~！

当你想使用 AI + E2E 后，第一件事就是——怎么让 AI 和浏览器交互。目前在这方面，业界没有任何的实践，**所有的胶水代码都必须由你来开发。**

目前 GPT 确实有检索和理解网页的能力，但是仅限于内容的理解。无法做到对于前端页面交互的理解，比如在选择框来选择某项在单元测试中，我们可以很容易的“报错后直接发给 AI，让 AI 多试几次”，但是这个交互流程在 E2E 情景下是没有相关实践的，**其中伴随很多的开发工作。**

同样的，AI 对于一个网页的理解是非常有限的，他并不知道一个用户交互是错误的还是正确的，你可能还需要传递给 AI 一些背景资料，比如产品文档、测试用例、交互图、代码。这部分的序列化工作也是一个全新的领域，**所有的胶水代码都必须由你来开发。**

如果你不是在一个大型 IT 公司，开发胶水代码成本是难以接受的。它可能要花费一年的时间，并且过程中没有任何阶段性产出，且部分功能随时可能被新一代的 LLM 所代替。

总结

通过单元测试 和 E2E 测试的分析，相信读者已经明白了的当前 AI & 测试面临的主要挑战：

- 1. 如何序列化让 AI 理解现状上下文
- 2. 过低接受率导致的额外人工介入的成本

但是我们相信这些这些问题都是有解的。可能是更加聪明的 LLM 提高了接受率，可能是规范的合作方合作流程，比如测试用例写的足够可读，并能直接导出为表格发送给 LLM，也可能是某个公司大笔一挥，决定花一年来开发一个 AI E2E 测试框架。

现在，软件测试的世界在等待一位英雄，他手持 LLM 的利剑，彻底斩除软件开发的质量痛点。而那位英雄，会不会是正在阅读本文的你呢？

AI & 辅助CR

除了开发与测试，换个角度看问题，**如果让 AI 来当我们的 CR 助手呢？**

代码审查（CodeReview）是软件开发过程中的重要环节，Code Review 是指开发人员对彼此的代码进行审查和评估的过程。通过 Code Review，团队成员可以相互检查代码的正确性、可读性、可维护性以及是否符合编码规范等方面的问题。可以帮助开发者发现并修复代码中的错误和缺陷，提高代码质量，减少技术债务。

传统的代码审查由人工完成，团队在 Code Review 上花费了大量时间，且容易出现疏漏。随着大型语言模型（LLM）的发展，LLM 在代码审查领域得到了越来越多的关注。LLM 可以通过对代码的语义理解，快速识别代码中的错误和缺陷，从而提高代码审查的效率和准确性，以下是 AI Code Review 的几个好处：

- **自动化和效率：**与传统的人工 Code Review 相比，AI Code Review 具有**更高的自动化程度**，可以大大提高审查的效率。
- **捕捉更多问题：**AI Code Review 工具可以检测出更多的代码问题，**包括潜在的性能问题、代码重复、安全漏洞等**，帮助开发人员更全面地改进代码。
- **减少人为偏差：**由于人为因素，传统的 Code Review 可能存在一些主观偏差。而 AI Code Review 可以基于大量的事实和规则进行评估，减少主观因素的影响，**提供更客观的评价结果。**
- **持续集成与部署：**AI Code Review 工具可以与**CI/CD 相结合**，实现自动的代码审查和反馈，帮助开发团队更好地管理代码质量。

AI 在 Code Review 的场景

LLM 在代码审查领域的应用主要有以下几个方面：

- **补充 CR 信息：**根据 CR 的 Diff 内容，生成 CR 的表述、标题、CR 总结，方便人工短时间了解代码变更关键信息。
- **基本代码问题：**变量名、函数名的语义化、可读性、可维护性、可扩展性等。通过分析代码的结构、语法和语义特征，工具可以自动判断代码的质量等级，为开发团队提供改进建议。
- **代码性能问题：**AI 代码评审工具可以分析代码的执行效率，发现潜在的性能瓶颈和优化点。例如，工具可以识别循环嵌套、冗余计算等问题，并提供优化建议，提高代码的执行效率。
- **逻辑问题：**比如边界问题、条件判断、循环控制的合理性。
- **代码安全问题：**AI 代码评审工具可以用于识别潜在的安全漏洞和风险。例如，工具可以分析代码中的敏感数据处理、输入验证、权限控制、SQL 注入，XSS 等方面，发现可能存在的安全问题，并提供修复建议。

当前应用

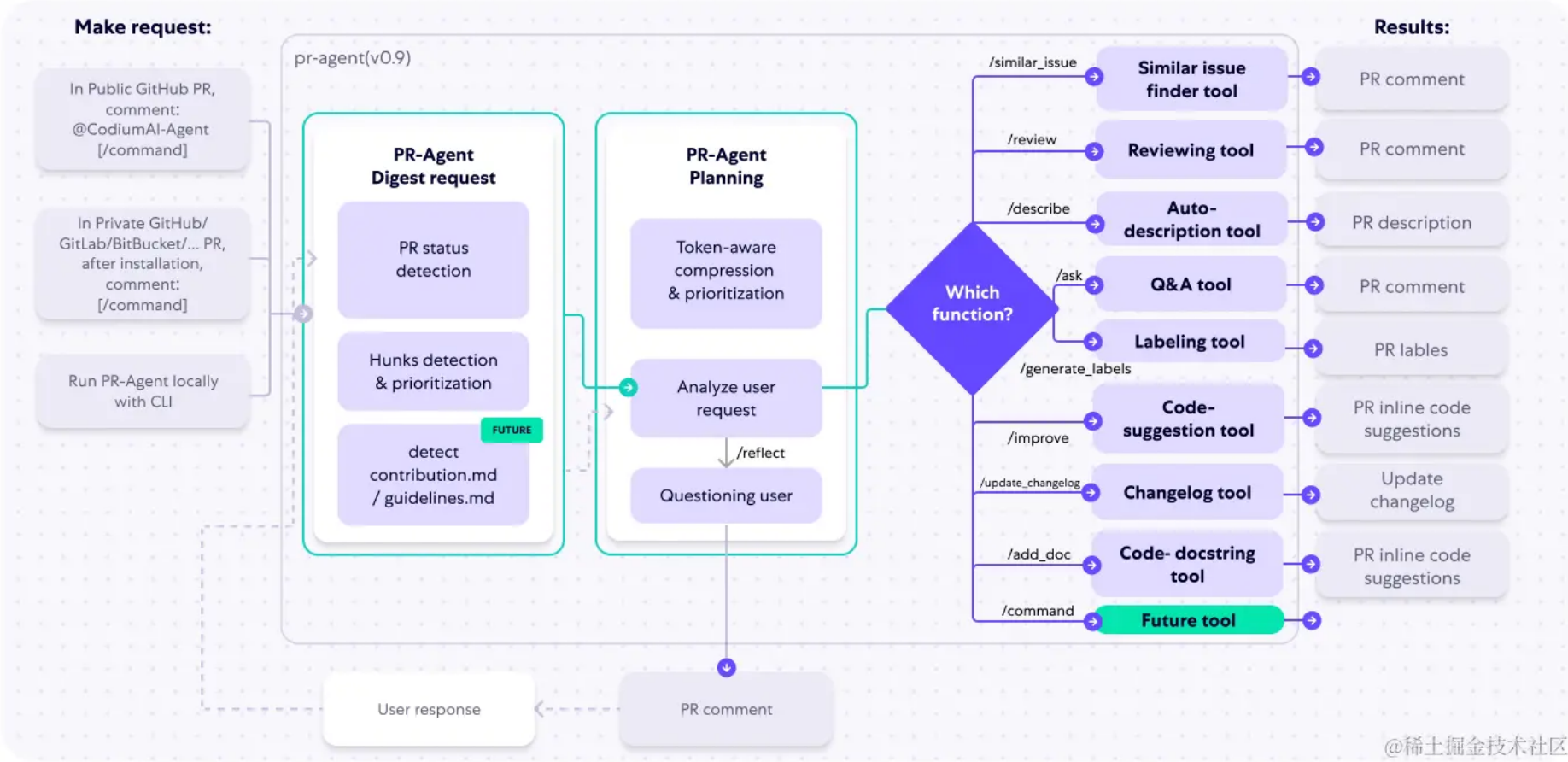
当前业界有不少 AI Code Review 的能力应用，除了大家熟知的 Github Copilot，开发者对其进行提问可以帮助完成 Code Review 的动作，我们再看一下还有哪些成熟的应用。

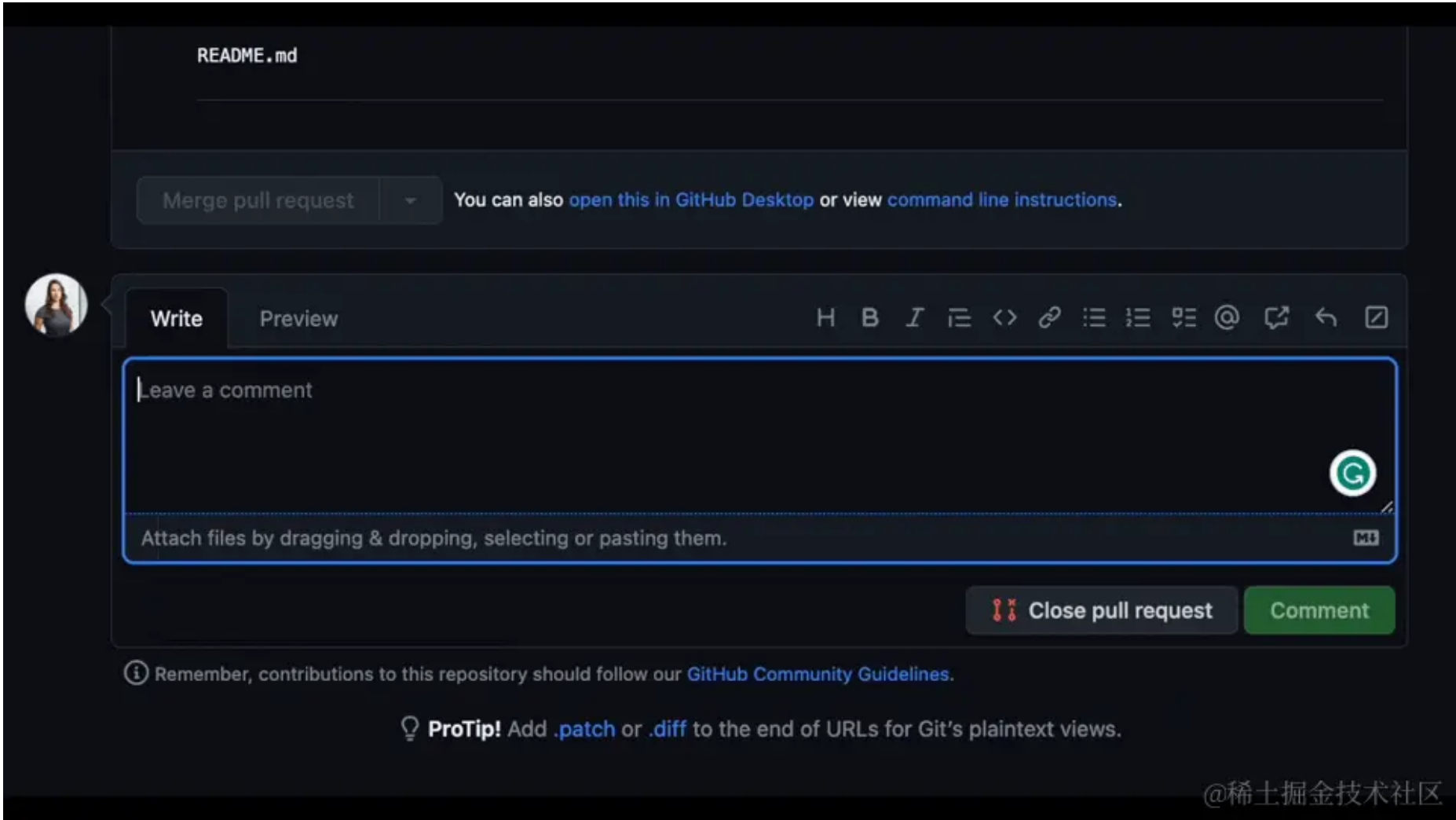
ChatGPT-CodeReview

通过调用 Chagpt 的 Open API 以 Github Action 的形式集成到 Github 里面，机器人会自动进行代码审查，审查信息将显示在 pr timeline / file changes 部分。在 `git push` 更新 PR 之后，CR bot 将重新审查更改的文件：

CodiumAI PR-Agent

CodiumAI PR-Agent：一款基于 Chatgpt 的开源工具，可帮助高效审查和处理拉取请求。它自动分析 pull request，并提供多种类型的命令：自动生成 PR 描述、可调整的反馈、问题解答、代码建议、更新 CHANGELOG.md 文件、查找相似问题、自动添加文档、生成自定义标签和自动分析 PR 变更。





总结

如果你决定开始使用 AI 能力作为团队 CR 助手，**请注意：**

- 由于 LLM 本身的 Token 上下文限制，**当 MR 超出最大的 Context 上下文限制时，会造成信息丢失导致 Review 结果不全面。**
- 由于 LLM 回答的随机性，同一份代码多次 Review 可能有不同的结果，以及 Review 的结论会偏啰嗦，不容易抓出重点问题，**所以在提示词里面可以多强调关注重点问题**，比如 Prompt 加入 “作为一名代码 review 专家、对 Gitlab 的 MR 进行代码 review。只关注代码性能、代码安全问题、严重的逻辑错误”
- 如果要针对团队业务场景或基于团队研发规范进行 AI Review，可以将团队的业务背景信息规范等作为知识库对模型进行微调，**对于担心代码泄漏的场景，还可以对开源的 LLM 进行私有化部署作为企业内的方案。**

一言以蔽之，当前只是辅助人工 Review 帮助快速发现一些问题，减轻人工的简单重复的劳动，但无法完全取代人，**最终还得由人作判断，但我们可以用节省下来的人力关注到更高优的事情上面。**

AI & 低代码

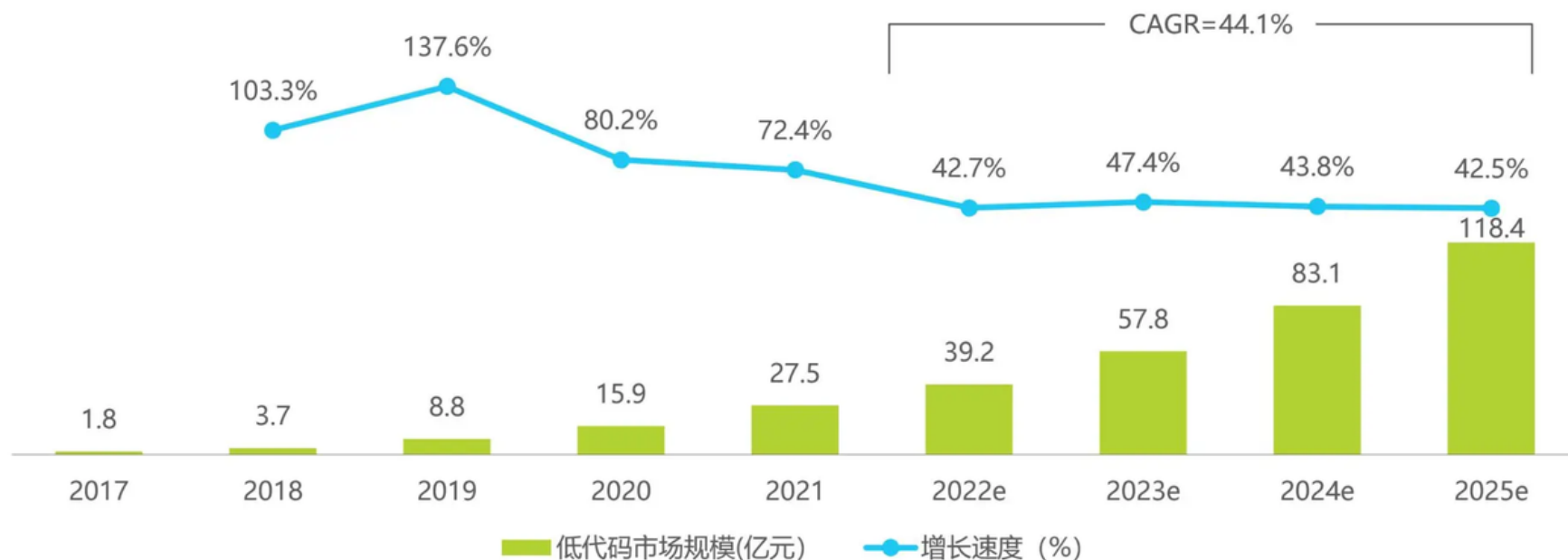
前两年火爆的低代码概念，似乎和 AI 有不错的“缘分”，两者结合往往能达到一加一大于二的效果。

什么是低代码

低代码开发平台（英语：Low-Code Development Platform，简称 LCDP），是一种**方便产生应用程序的平台软件**，软件会让用户以图形化接口以及配置编写程序，而不是用传统的程序设计作法。此平台是针对某些种类的应用而设计开发的，例如数据库、业务过程、以及用户界面（例如网页应用程序）。**这类平台一般可以产生完整且可运作的应用 代码，但在一些特殊的情形下仍需要编写程序。**

低代码概念最早出现在 1982 年《无程序员的应用程序开发》一书中，美国低代码产品的研究和实践过程较长，积累了较丰富的经验。中国最早出现低代码平台大概是在 2014 年，产品的形态从最初的数据库交付，到数据集结构搭建逐渐演变抽象出各种流程引擎、可视化界面等产品能力，应用能力也从 BPM 延伸到 ERP、CRM 等大型复杂应用，整个行业经历了 2017-2020 年的快速发展阶段，市场增速开始放缓，**但相对于其他企业服务赛道仍处于高增长阶段，预计 2025 年中国低代码行业市场规模将达到 118.4 亿。**

2017-2025年中国低代码行业市场规模及增速



注释：本报告市场规模统计仅包含通过提供独立低代码平台并产生营收的厂商，低代码平台仅供内部产品使用的部分未包含在本报告研究范围。
来源：根据专家访谈、公司财报以及艾瑞市场模型推算获得，艾瑞咨询研究院自主研究及绘制。

©2022.8 iResearch Inc.

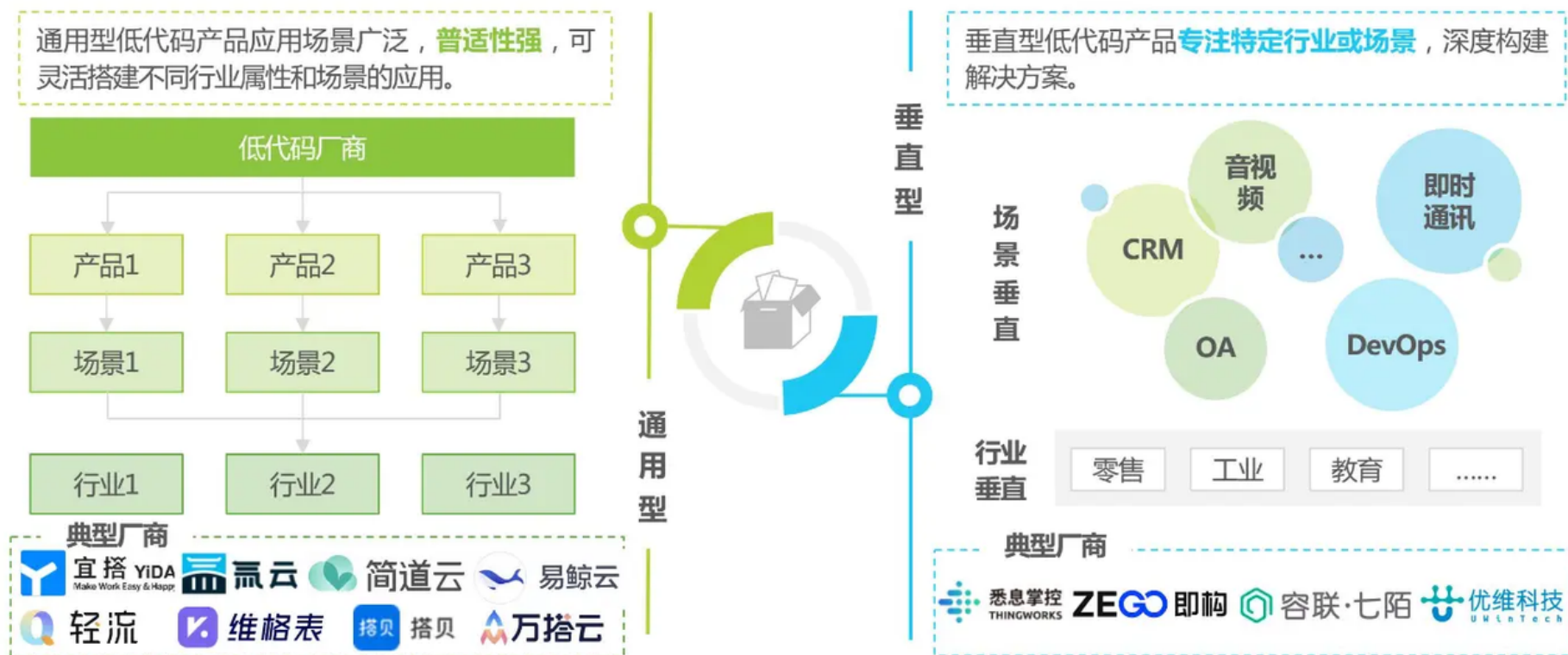
www.iresearch.com.cn
@稀土掘金技术社区

存在问题

但低代码自出现以来，一直还是饱受质疑，核心的点在于通过低代码的可视化搭建大大降低了建设成本，且使用群体从开发人员逐步向业务人员渗透，但低代码本身是存在一定的上手门槛，包括各类物料的用法、如何实现与后端接口通信、物料之间如何配置交互等等，这使得业务人员无法快速上手完成页面搭建，而开发人员需要学习特定搭建规则和限制，可能不如实际开发来的快。

在不同类型的低代码产品的相关的限制又会有所差异，从应用场景上可以划分通用型和垂直型

通用型和垂直型产品特征对比



来源：艾瑞咨询研究院自主研究及绘制。

©2022.3 iResearch Inc.

www.iresearch.com.cn
@稀土掘金技术社区

- 对于垂直型低代码产品，深耕特定行业或特定的场景，提供当前垂类更具有专业性的解决方案和搭建能力，在具体的搭建场景中专业性贴合更深，会存在复杂搭建场景，如公式、模型、数据等，门槛更高，搭建依赖的前置输入也要更多。
- 而对于通用型，由于覆盖各行业各场景，沉淀了大量的行业模版和解决方案，虽然搭建门槛相对垂直型要更低，但内容覆盖面更广，大量的物料和模版不仅造成了用户选择压力，也提升了搭建的学习负担。

低代码产品本身的易用性有待进一步提升，包括底层技术框架和可视化界面的呈现，另外低代码相关的培训机制有待完善，培训不足导致用户对于低代码产品的接纳度低，用户本身对于繁重的产品使用学习的意愿不足，也会大大阻碍低代码产品的推广和应用。

总结

当然 AIGC 与低代码的融合还处于早期阶段，当前应用范围也主要还是在一些容错率较高的场景，通过大语言模型可能不能完全理解用户的真实需求，这需要庞大的上下文做支撑，虽然融合方向仍然有诸多问题亟需解决，但未来低代码平台必将会全面拥抱 AIGC，这也意味着我们需要重新去思考低代码平台的交互界面的设计，充分利用自然语言交互特点，提升用户体验。

另外数据安全风险也是在商业化产品中必须要考量的一点，因此多数企业都在尝试走自研大模型的路子，然而这并不是一件容易的事。它需要大量高质量数据的收集、清洗和标注，以及昂贵的计算资源来支持模型训练。这也意味着未来 AI 加持下的低代码平台的竞争成本会更高，会更加聚焦在有雄厚实力的头部企业。

除了 AIGC +低代码融合之外，还有一方观点认为既然 AIGC 能力越来越强，是否会完全替代低代码，直接生成原始应用？我们认为至少在可以预见很长范围内低代码不会被完全替换掉，就像上面几部分提到的，大模型通过语义直接生成代码并不能保证其完全的准确性，且代码可用性需要大量的人工校正。而低代码融合 AIGC 可以通过语义生成模型，加速需求分析工作进程，可以让 AI 更快的进入业务。

6. 师资&课程内容

1. 师资：一线大厂/外企现役P8级高级专家亲自带；
2. 内容：深度对标阿里P6-P7，涵盖大厂所有内训标准；

6.1 项目实战内容

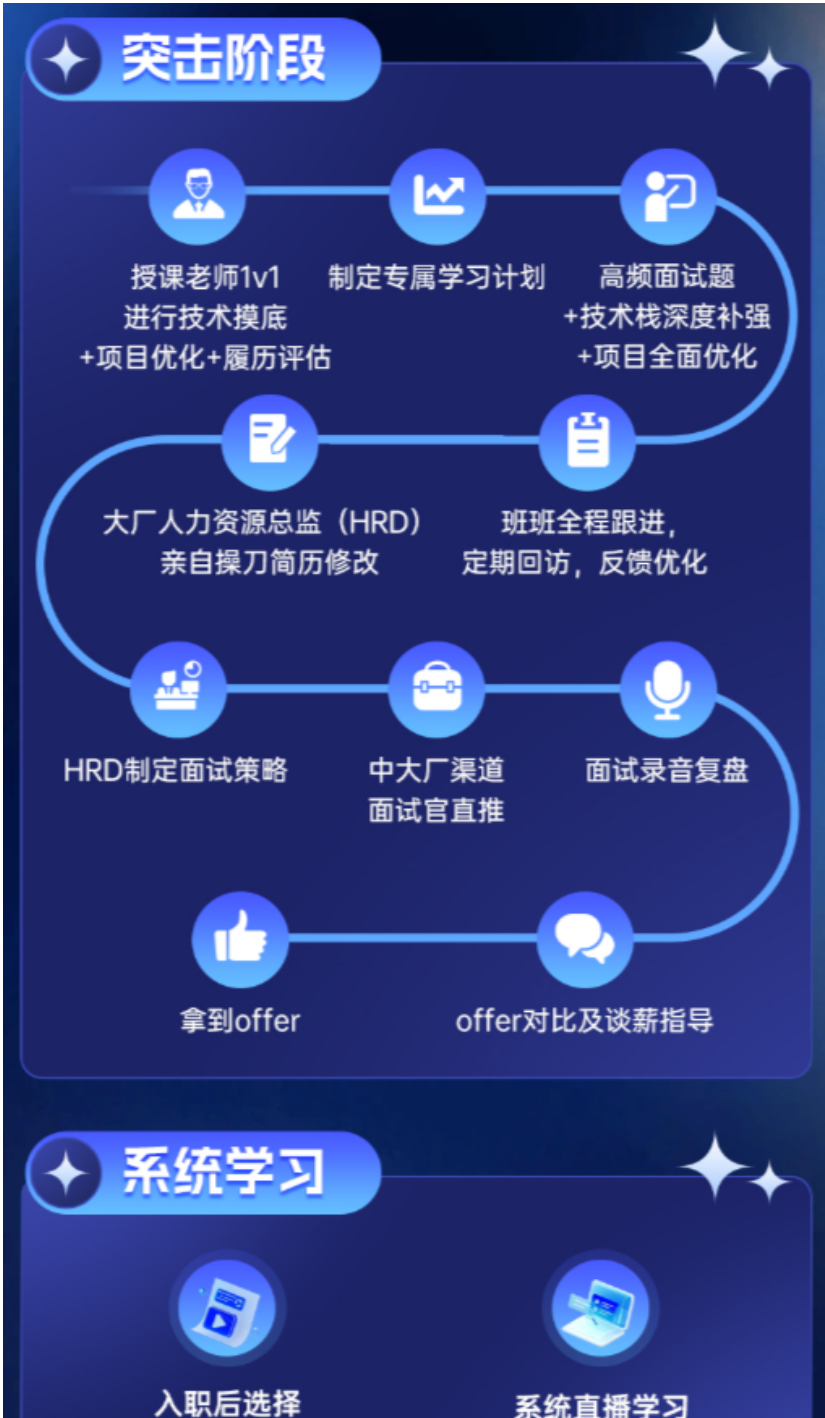
github：<https://github.com/encode-studio-fe>

[目前端实战课分享汇总](#)（可咨询班主任要密码权限）

6.2 服务内容

系统&突击两手抓

全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



6.3 专业一对一

近期一对一

- 目 (2025-02-06) xxx-2.5年-React-24K
- 目 (2025-02-07) xxx-1年-React-15k
- 目 (2025-01-05) xxx-7年-Vue-20k

报名学员专享

目 13. 如何写出高质量的简历 (For 付费学员)

6.4 渠道内推 & 保障就业

渠道：面试官直推/高管直推 (简历直达leader)

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

6.5 近期报名学员案例

